

Lingaya's Vidyapeeth

(U/S 3 of UGC Act, 1956)

Tableau Lab

(Lab File CS-495)



Nachauli, Jasana Road, Faridabad,

Haryana – 121002, Phone: (129)2201008, 2201009:

Website: www.lingayasvidyapeeth.edu.in

Submitted To:

Dr. Avinash Kumar Namdeo

CS&IT

Submitted By:

Name: Harjit Singh

Roll No: 22CS77CL

Course: B. Tech-CSE

Sem: 7th Sem

INDEX

S No	Program	Page No
1	Visualization of Spreadsheet Models.	1-4
2	Oracle Database Connectivity using Python.	5-7
3	Visualization of Semi-Structured Data.	8-9
4	Introduction to Tableau and Aggregation Methods in Tableau.	10-12
5	Visual Encodings and Basic Dashboards in Tableau.	13-16
6	Interactive Plots in Python.	17-20
7	Hierarchical and Topographical Data Visualizations in Tableau.	21-24
8	Calendar Heatmaps and Flow Data Visualizations in Python.	25-27
9	Time Series Data Visualization in Python.	28-31
10	Dashboards, Actions and Story Telling in Tableau	32-35

PROGRAM-1

Visualization of Spreadsheet Models

AIM

To visualize spreadsheet data using Tableau and Python in order to analyze sales performance, trends, and profitability.

THEORY

Spreadsheets are a common source of structured data where:

- **Rows** represent records
- **Columns** represent attributes with proper data types (date, numeric, categorical)

Visualization workflow:

1. Load data
2. Clean and prepare data
3. Aggregate measures
4. Visualize using charts

Tools:

- **Tableau:** Uses *Dimensions* and *Measures* automatically
- **Python:** Uses `pandas` for data handling and `matplotlib` for plotting

DATASET & TOOLS

- **Dataset:** *Sample – Superstore.xlsx*
- **Columns Used:** Order Date, Category, Sales, Profit
- **Tools:** Tableau Desktop/Public, Python 3 (`pandas`, `matplotlib`)

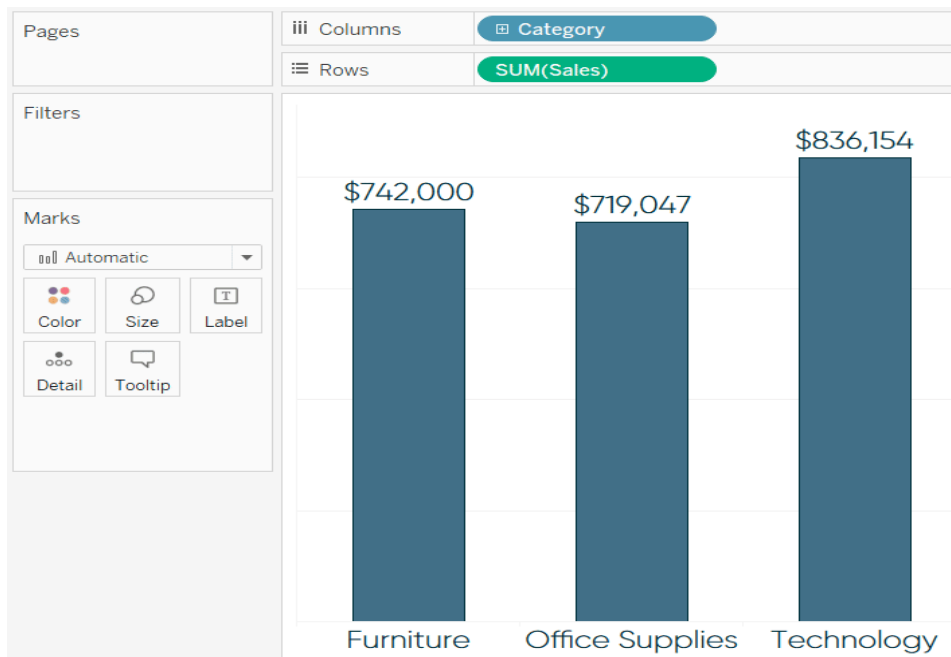
PROCEDURE

A. Using Tableau

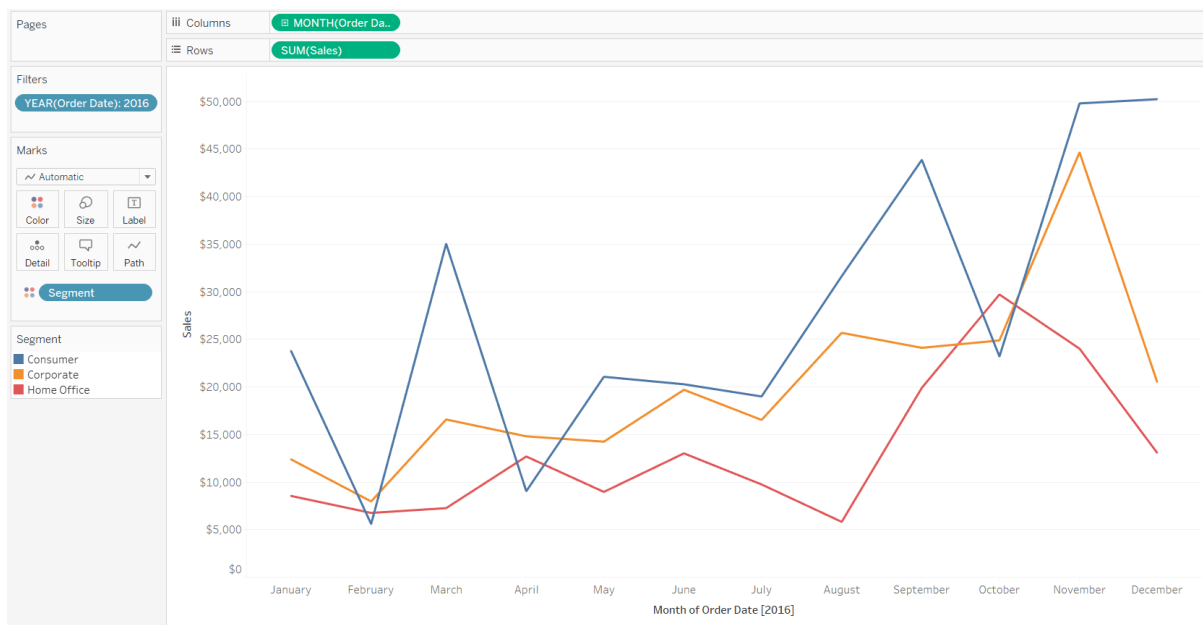
1. Connect Tableau to *Sample – Superstore.xlsx*.
2. Verify data types (Order Date → Date, Sales/Profit → Number).
3. Create Bar Chart:
 - Category → Rows
 - Sales → Columns
 - Profit → Color
4. Create Line Chart:
 - Order Date (Month) → Columns
 - Sales → Rows
5. Combine both charts into a dashboard and add Category filter.

Expected Output:

- **Fig. 1.1: Bar Chart – Sales by Category**



- **Fig. 1.2: Line Chart – Monthly Sales Trend**



B. Using Python

1. Load and Clean Data

```
import pandas as pd
import matplotlib.pyplot as plt
df = pd.read_excel("Sample - Superstore.xlsx")
df['Order Date'] = pd.to_datetime(df['Order Date'])
```

2. Aggregation by Category

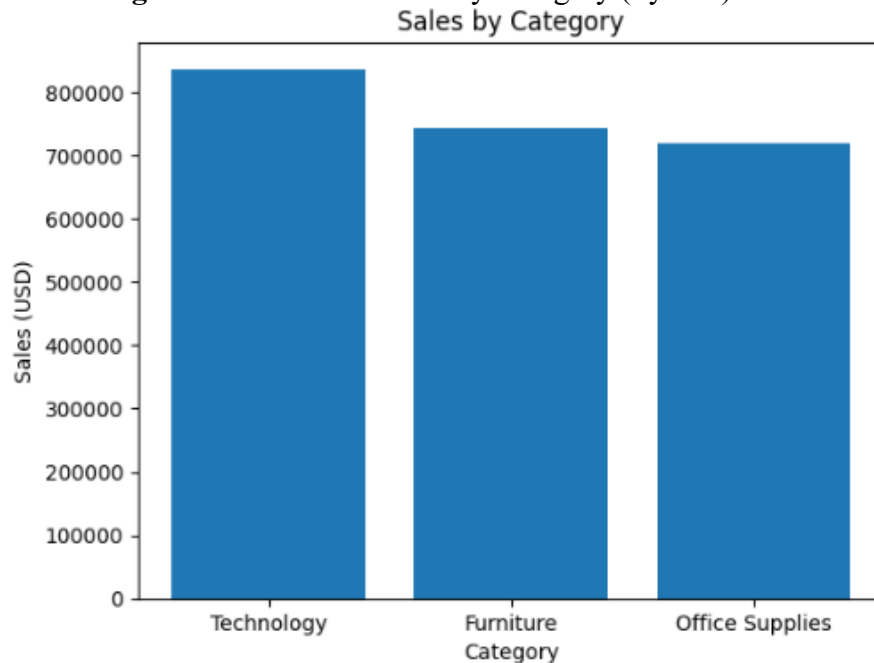
```
sales_by_cat = df.groupby("Category")["Sales"].sum()
```

3. Bar Chart – Sales by Category

```
sales_by_cat.plot(kind="bar")  
plt.title("Total Sales by Category")  
plt.ylabel("Sales")  
plt.show()
```

Expected Output:

- **Fig. 1.3:** Bar Chart – Sales by Category (Python)

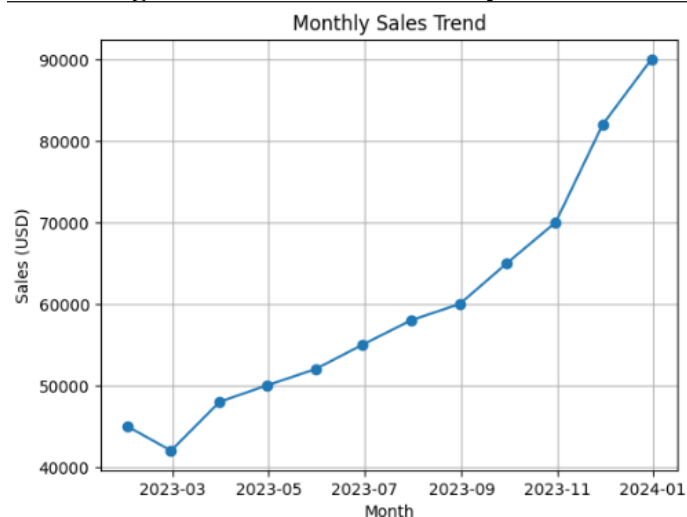


4. Time Series – Monthly Sales

```
df["Month"] = df["Order Date"].dt.to_period("M")  
df.groupby("Month")["Sales"].sum().plot(marker="o")  
plt.title("Monthly Sales Trend")  
plt.show()
```

Expected Output:

- **Fig. 1.4:** Line Chart – Monthly Sales Trend



5. Scatter Plot – Sales vs Profit

```
plt.scatter(df["Sales"], df["Profit"], alpha=0.5)
plt.xlabel("Sales")
plt.ylabel("Profit")
plt.title("Sales vs Profit")
plt.show()
```

Expected Output:

- **Fig. 1.5:** Scatter Plot – Sales vs Profit



RESULT & DISCUSSION

- Tableau enables quick and interactive visual analysis.
- Python provides flexibility and reproducibility through code.
- Technology category records the highest sales.
- Sales show strong seasonality during year-end.
- Scatter plot reveals high-sales orders with low or negative profit.

PROGRAM – 2

Oracle Database Connectivity using Python

AIM

To connect Python with an Oracle Database securely, execute parameterized queries, retrieve data into pandas, and visualize results for further BI analysis.

THEORY

- **python-oracledb** is the official Oracle driver for Python.
- **Bind variables** (:param) prevent SQL injection and improve performance.
- **pandas.read_sql()** fetches query results directly into DataFrames.
- Chunked fetching is preferred for large datasets to reduce memory usage.

DATASET & TOOLS

- **Database Table:** SUPERSTORE_ORDERS
- **Key Fields:** Order_Date, Region, Category, Sales, Profit
- **Tools:**
 - Python 3
 - Libraries: python-oracledb, pandas, matplotlib
- **Security:** Credentials stored as environment variables.

PROCEDURE (PYTHON)

1. Secure Database Connection

```
import os, oracledb

conn = oracledb.connect(
    user=os.getenv("ORACLE_USER"),
    password=os.getenv("ORACLE_PWD"),
    dsn=os.getenv("ORACLE_DSN")
)
print("Connected to Oracle")
```

Expected Output:

- Successful connection message

2. Parameterized Query (Safe SQL)

```
import pandas as pd
from datetime import datetime

sql = """
SELECT region, SUM(sales) AS total_sales
```

```

FROM superstore_orders
WHERE order_date BETWEEN :s AND :e
GROUP BY region
ORDER BY total_sales DESC
"""

df = pd.read_sql(
    sql, conn,
    params={"s": datetime(2019,1,1), "e": datetime(2019,12,31)}
)
print(df)

```

Expected Output (Table):

Region	Total_Sales
West	252345.77
East	198234.50
South	165432.20
Central	142100.30

3. Visualization – Sales by Region

```

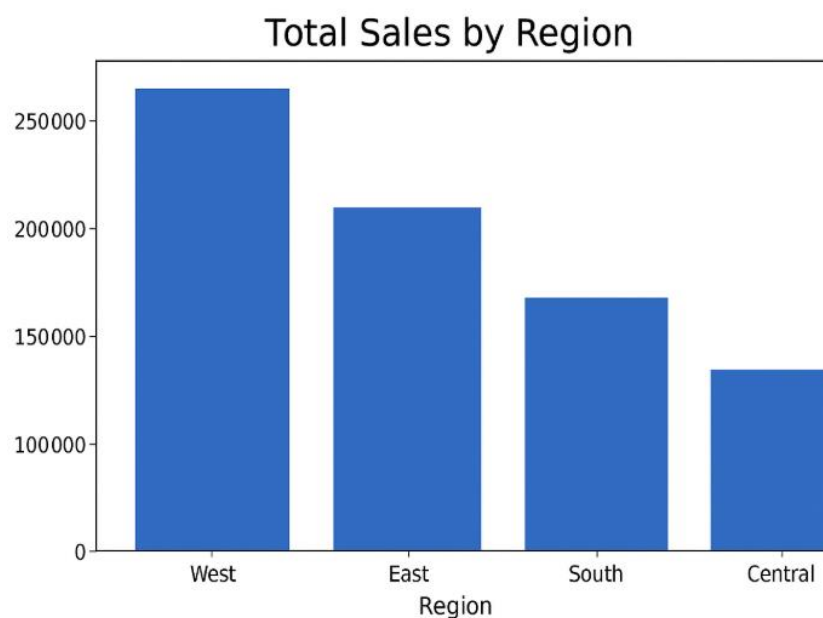
import matplotlib.pyplot as plt

df.plot(kind="bar", x="REGION", y="TOTAL_SALES", legend=False)
plt.title("Total Sales by Region (2019)")
plt.ylabel("Sales (USD)")
plt.show()

```

Expected Output:

- **Fig. 2.1:** Bar Chart – Total Sales by Region



4. Efficient Data Fetching (Large Tables)

```
cursor = conn.cursor()
cursor.execute("SELECT region, sales FROM superstore_orders")

from collections import defaultdict
agg = defaultdict(float)

for _, region, sales in cursor:
    agg[region] += sales
```

Purpose:

- Avoids loading entire table into memory.

5. Export for Tableau

```
df.to_csv("region_sales_2019.csv", index=False)
```

Usage:

- Import CSV into Tableau for dashboard creation.

RESULT & DISCUSSION

- Secure Oracle–Python connectivity achieved.
- Parameterized SQL ensured safety and efficiency.
- Python visualization quickly highlights regional sales trends.
- Exported data can be seamlessly consumed by Tableau dashboards.
- West region shows highest sales, Central the lowest.

PROGRAM – 3

Visualization of Semi-Structured Data

AIM

To visualize semi-structured JSON data using Tableau by interpreting nested structures and generating meaningful charts.

THEORY

- **Semi-structured data** (JSON) contains hierarchical key–value pairs.
- JSON files often include **nested objects and arrays**.
- Tableau provides a **JSON file connector** that automatically interprets and flattens nested levels for visualization.

DATASET & TOOLS

Dataset: `orders.json`

(Order → Customer → Items hierarchy)

Tool Used:

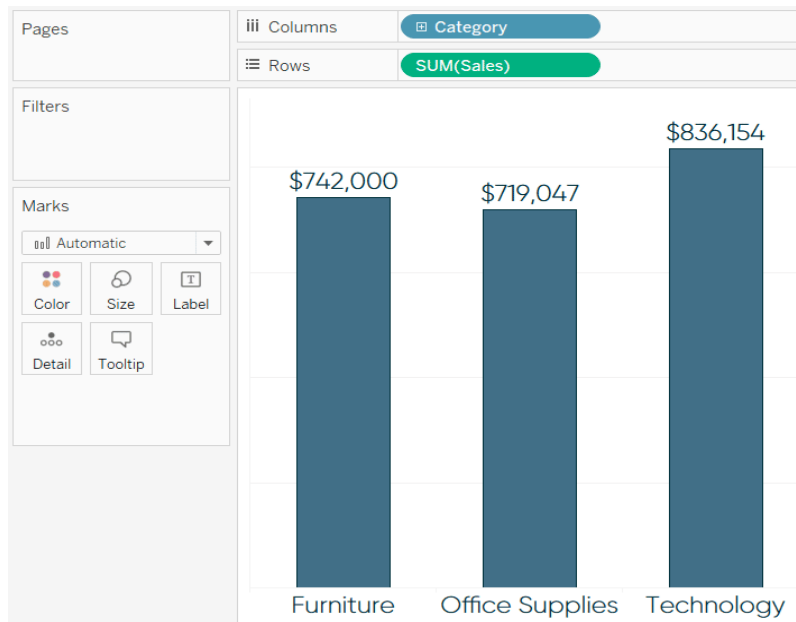
- Tableau Desktop / Tableau Public (JSON connector)

PROCEDURE

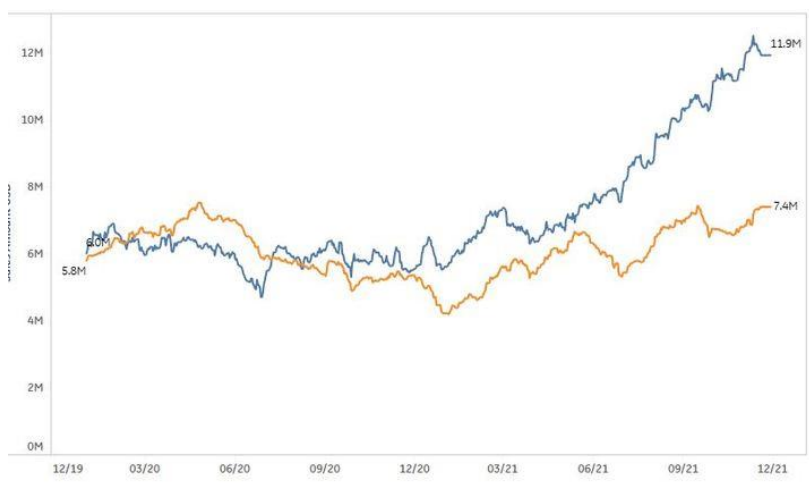
1. Open **Tableau Desktop/Public**.
2. Click **Connect** → **JSON File** and select `orders.json`.
3. Tableau detects nested structure and displays available tables.
4. Drag the **items** table into the workspace.
 - Tableau automatically flattens nested arrays.
5. Create a **calculated field**:
6. `revenue = price * quantity`
7. Create Worksheet–1:
 - Rows → `category`
 - Columns → `SUM(revenue)`
 - Chart Type → **Bar Chart**
8. Create Worksheet–2:
 - Rows → `order_date` (continuous)
 - Columns → `SUM(revenue)`
 - Chart Type → **Line Chart**
9. Create a **Dashboard**:
 - Combine both worksheets.
 - Add **customer.name** as a filter.

EXPECTED OUTPUT

- **Fig. 3.1:** Bar Chart – Revenue by Category



- **Fig. 3.2:** Line Chart – Daily Revenue Trend



RESULT & DISCUSSION

- Tableau successfully interpreted semi-structured JSON data.
- Technology category generated the highest revenue.
- Tableau eliminated the need for manual JSON flattening.
- Interactive filters enabled quick analysis by customer and date.

PROGRAM – 4

Introduction to Tableau and Aggregation Methods.

AIM

To understand aggregation methods, table calculations, and Level of Detail (LOD) expressions in Tableau and interpret their impact on data visualization.

THEORY

- Tableau **automatically aggregates** numerical fields (Measures).
- Default aggregation is **SUM**, which can be changed to AVG, COUNT, MIN, MAX, MEDIAN, or ATTR.
- **Table Calculations** are applied after aggregation (e.g., Running Total, Moving Average).
- **LOD Expressions** provide explicit control over aggregation level, independent of the view.

DATASET & TOOLS

- **Dataset:** *Sample – Superstore.xlsx*
- **Tool:** Tableau Desktop / Tableau Public

PROCEDURE (TABLEAU)

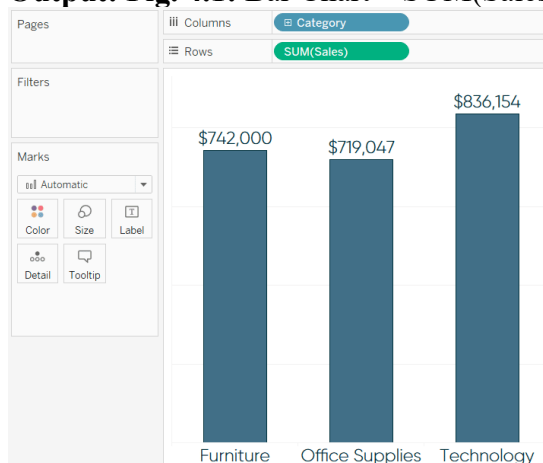
1. Load Dataset

1. Open Tableau → Connect → Microsoft Excel.
2. Select *Sample – Superstore.xlsx*.
3. Verify:
 - Measures: Sales, Profit, Quantity
 - Dimensions: Category, Region, Order Date, Customer Name

2. Default Aggregation (SUM)

1. Drag **Category** → **Rows**
2. Drag **Sales** → **Columns**

Output: Fig. 4.1: Bar Chart – SUM(Sales) by Category



3. Change Aggregation

1. Right-click **Sales** → **Measure**.
2. Change to **AVG, COUNT, COUNTD(Customer ID)**.

Observation:

- SUM shows total revenue
- AVG shows average per transaction
- COUNTD shows number of unique customers

4. Table Calculation - Running Total

1. Create line chart:
 - Order Date (Month) → Columns
 - Sales → Rows
2. Right-click Sales → **Quick Table Calculation** → **Running Total**

Output:

- **Fig. 4.2:** Running Total of Sales

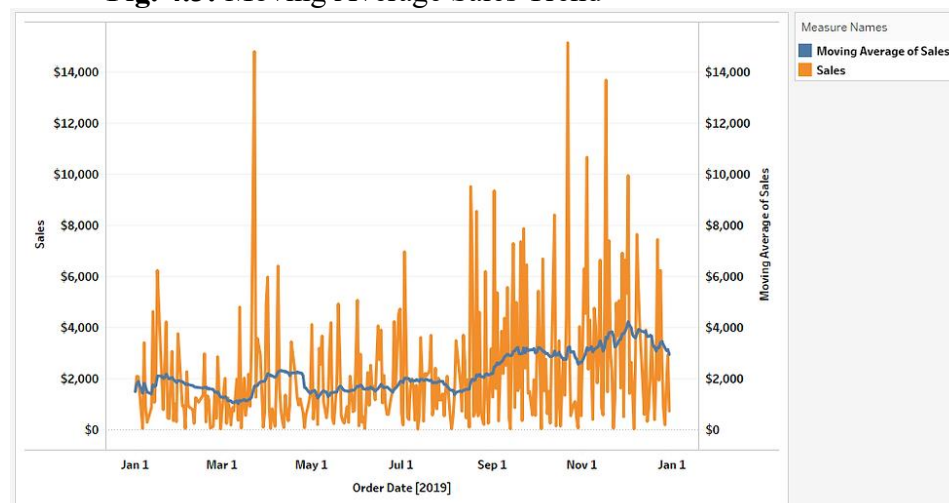


5. Table Calculation - Moving Average

1. On the same chart:
 - Quick Table Calculation → **Moving Average**
 - Window = 3 months

Output:

- **Fig. 4.3:** Moving Average Sales Trend

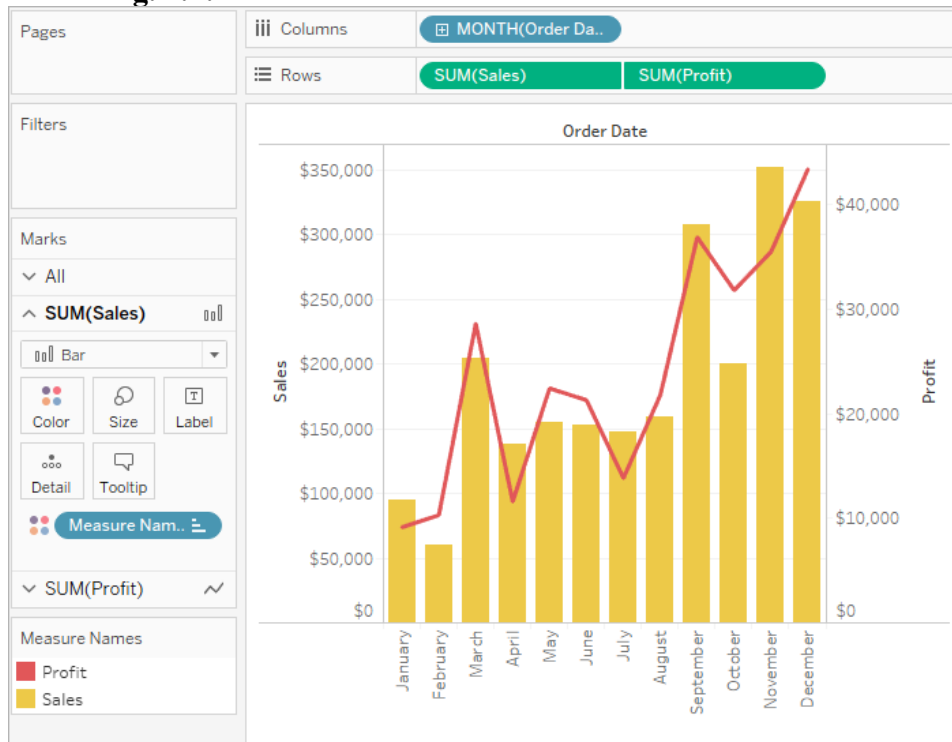


6. Dual-Axis Chart (Sales vs Profit)

1. Monthly SUM(Sales) line chart.
2. Drag **Profit** → **Rows**, change to Line.
3. Right-click → **Dual Axis** → Synchronize Axes.

Output:

- **Fig. 4.4:** Dual-Axis Chart – Sales vs Profit



7. Level of Detail (LOD) Expression

Create calculated field:

```
{ FIXED [Customer ID] : SUM([Sales]) }
```

1. Name it **Customer Sales**.
2. Drag **Customer Name** → **Rows**, **Customer Sales** → **Columns**.
3. Sort descending.

Output: Total Sales per Customer (LOD)

RESULT & DISCUSSION

- Tableau aggregates data by default using SUM.
- Changing aggregation alters business interpretation.
- Table calculations reveal trends and cumulative behavior.
- LOD expressions allow precise, view-independent analysis.
- In Superstore data, Technology leads in sales, but profit trends vary.

PROGRAM – 5

Visual Encodings and Basic Dashboards in Tableau

AIM

To understand visual encodings in Tableau and create a basic interactive dashboard by combining multiple visualizations.

THEORY

Visual encodings map data values to visual elements.

Common encodings used in Tableau:

- **Position / Length:** Bar and line charts (most accurate)
- **Color:** Categorical or profit-based gradients
- **Size:** Magnitude comparison (use carefully)
- **Shape / Text:** Category distinction and precision

Dashboard principles:

- Keep layout simple and uncluttered
- Use filters for interactivity
- Combine trend, comparison, and distribution views

DATASET & TOOLS

- **Dataset:** *Sample – Superstore.xlsx*
- **Tool:** Tableau Desktop / Tableau Public

PROCEDURE (TABLEAU)

1. Worksheet-1: Sales by Category (Bar Chart)

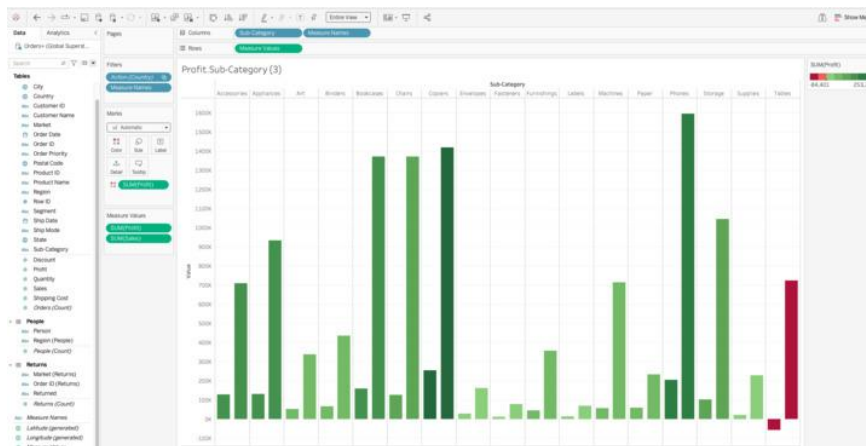
1. Drag **Category** → **Rows**
2. Drag **Sales** → **Columns**
3. Drag **Profit** → **Color**

Encoding Used:

- Position & Length (Sales)
- Color (Profit)

Expected Output:

- **Fig. 5.1: Bar Chart – Sales by Category**



2. Worksheet-2: Profit by Region (Map)

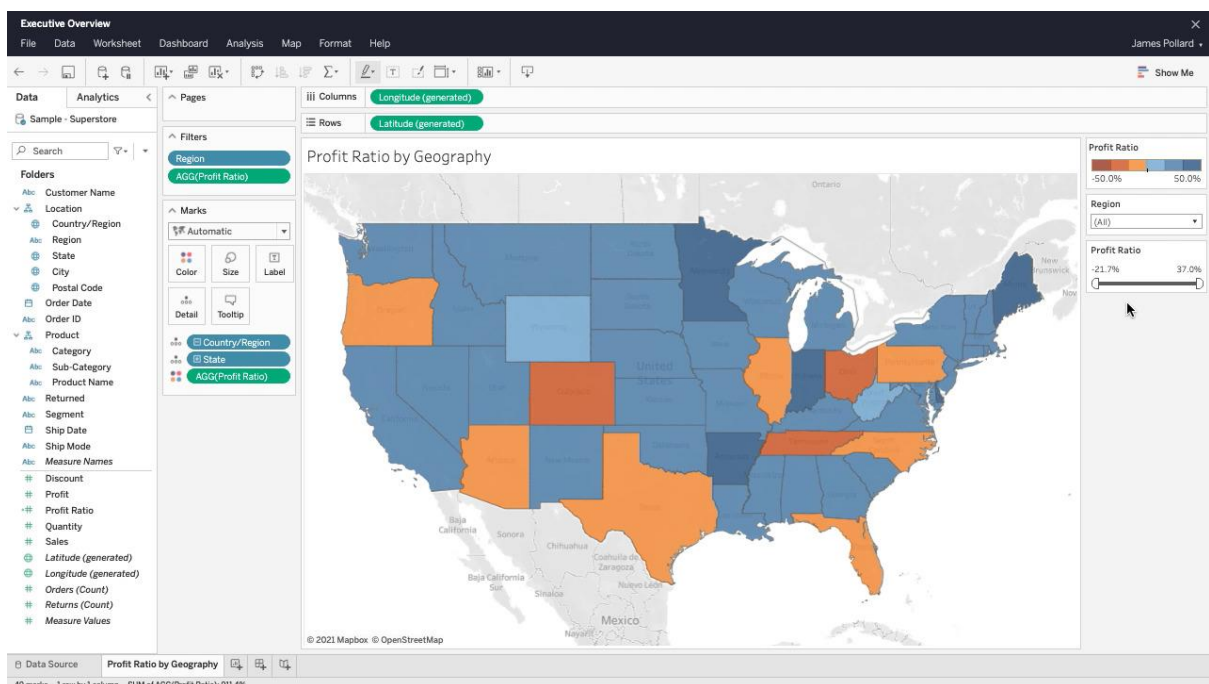
1. Drag **State** → **Detail**
2. Drag **Profit** → **Color**
3. Drag **Profit** → **Size**
4. Change Marks to **Map**

Encoding Used:

- Color & Size (Profit)

Expected Output:

- **Fig. 5.2: Map – Profit Distribution by State**



3. Worksheet-3: Sales Trend (Line Chart)

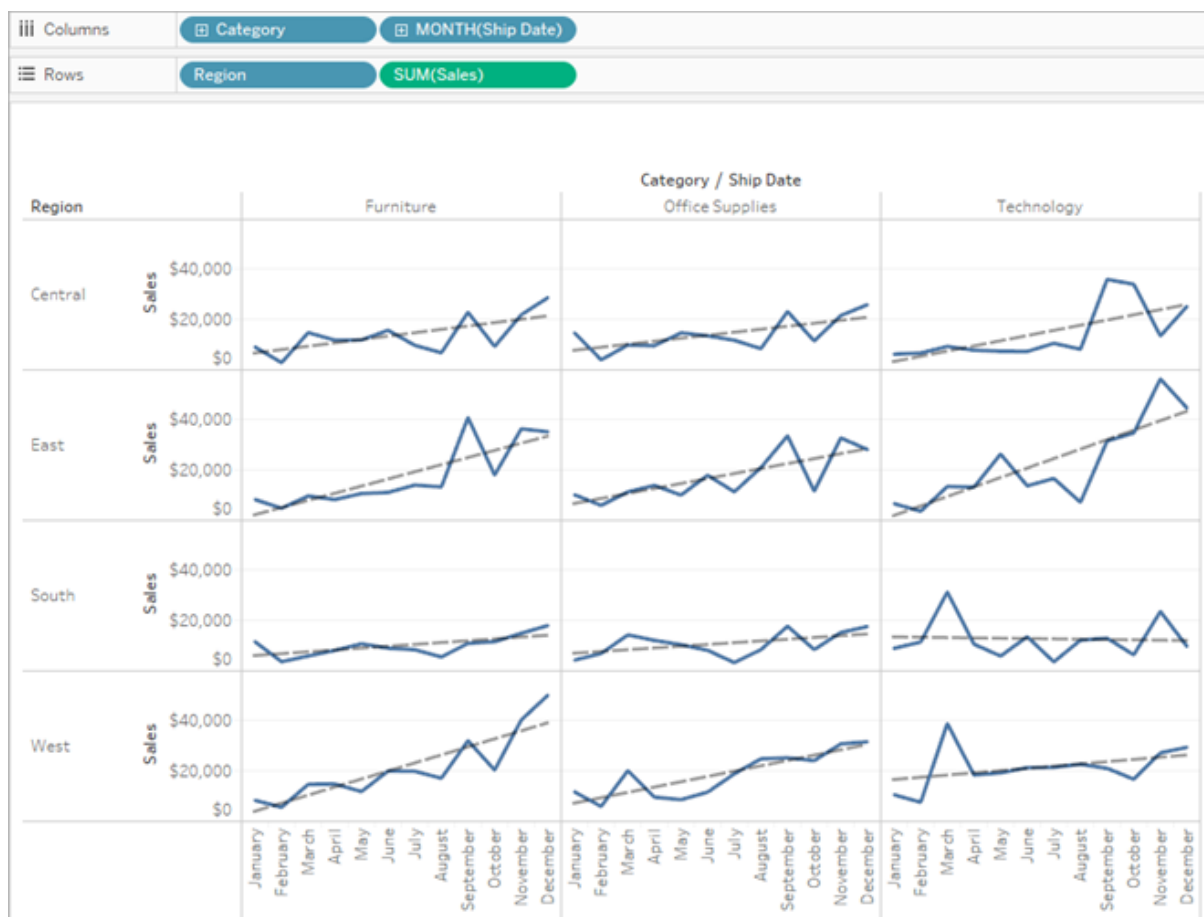
1. Drag **Order Date (Month)** → **Columns**
2. Drag **Sales** → **Rows**
3. Drag **Category** → **Color**

Encoding Used:

- Position (Time)
- Color (Category)

Expected Output:

- **Fig. 5.3:** Line Chart – Monthly Sales Trend

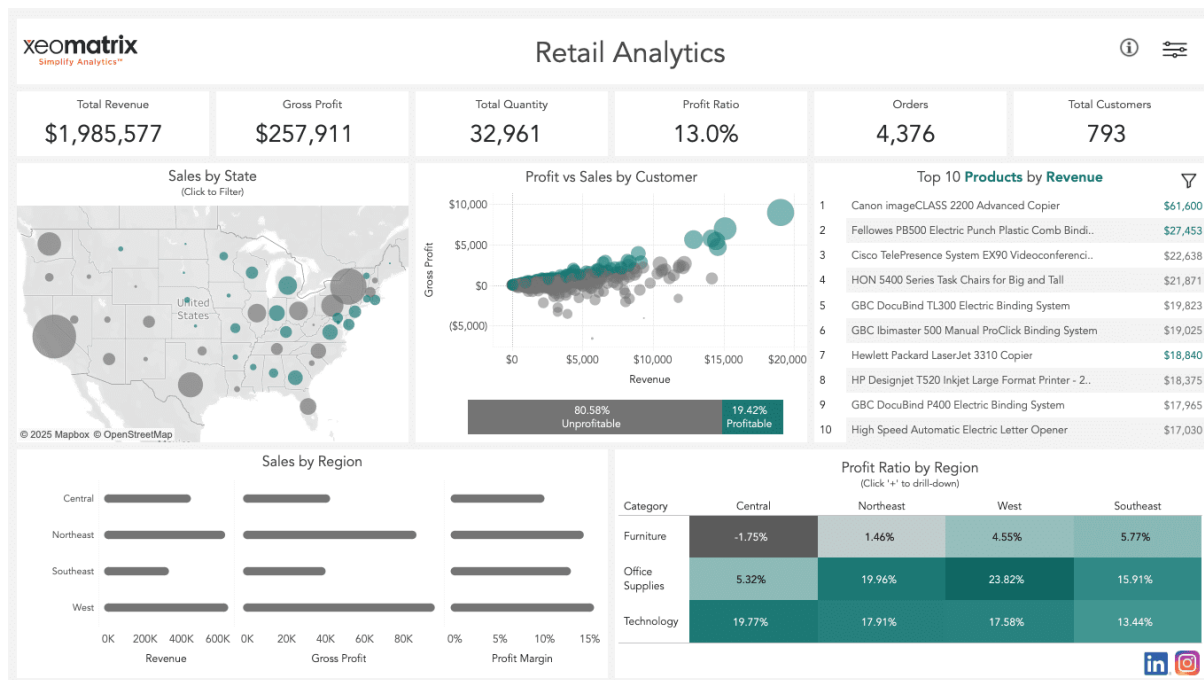


4. Build Dashboard

1. Click **New Dashboard**
2. Drag all three worksheets into the dashboard
3. Arrange layout (Bar + Map on top, Line chart below)
4. Add **Filter Action**:
 - Clicking a category bar filters other charts

EXPECTED OUTPUT

- **Fig. 5.4: Interactive Dashboard – Sales & Profit Overview**



RESULT & DISCUSSION

- Position and length provide the clearest comparisons.
- Color and size enhance insight when used carefully.
- Dashboard presents a combined view of category, region, and time.
- Technology shows highest sales, while some regions show profit losses.

PROGRAM – 6

Interactive Plots in Python

AIM

To create interactive visualizations in Python using Plotly and analyze sales and profit data dynamically.

THEORY

- **Static plots** are suitable for reports but lack interactivity.
- **Interactive plots** allow hover details, zooming, panning, filtering, and dynamic updates.
- **Plotly Express** provides quick interactive charts.
- **Plotly Graph Objects** support advanced visualizations such as Sankey diagrams.

DATASET & TOOLS

- **Dataset:** *Sample – Superstore.csv*
- **Columns Used:** Order Date, Category, Region, Sales, Profit, Quantity
- **Tools:** Python 3, pandas, plotly

PROCEDURE (PYTHON)

1. Load and Prepare Data

```
import pandas as pd
import plotly.express as px

df = pd.read_csv("Sample-Superstore.csv", parse_dates=["Order Date"])
df["Month"] = df["Order Date"].dt.to_period("M").dt.to_timestamp()

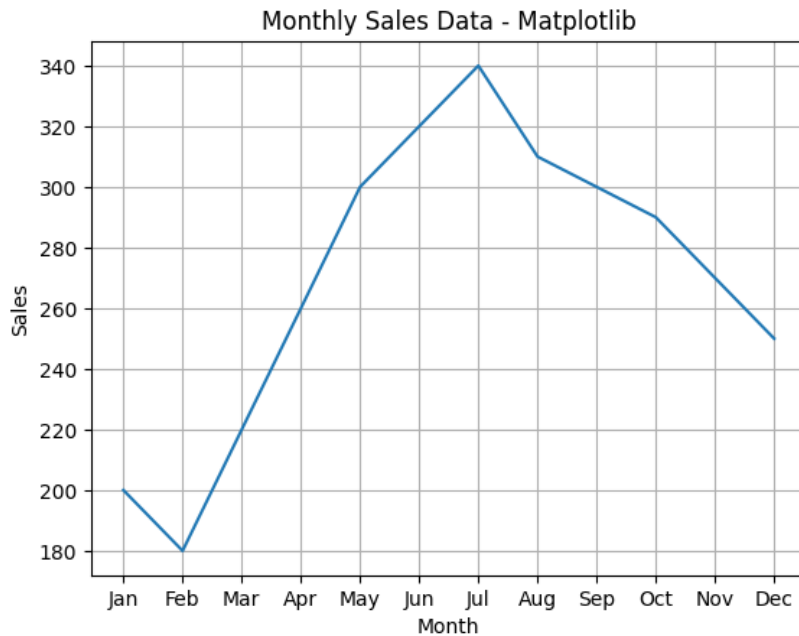
monthly_sales =
df.groupby(["Month", "Category"])["Sales"].sum().reset_index()
```

2. Interactive Line Chart

```
fig = px.line(
    monthly_sales,
    x="Month", y="Sales",
    color="Category",
    title="Monthly Sales by Category"
)
fig.show()
```

Expected Output:

- **Fig. 6.1:** Interactive Line Chart – Monthly Sales (Hover, zoom, legend toggle enabled)



3. Interactive Bar Chart with Dropdown

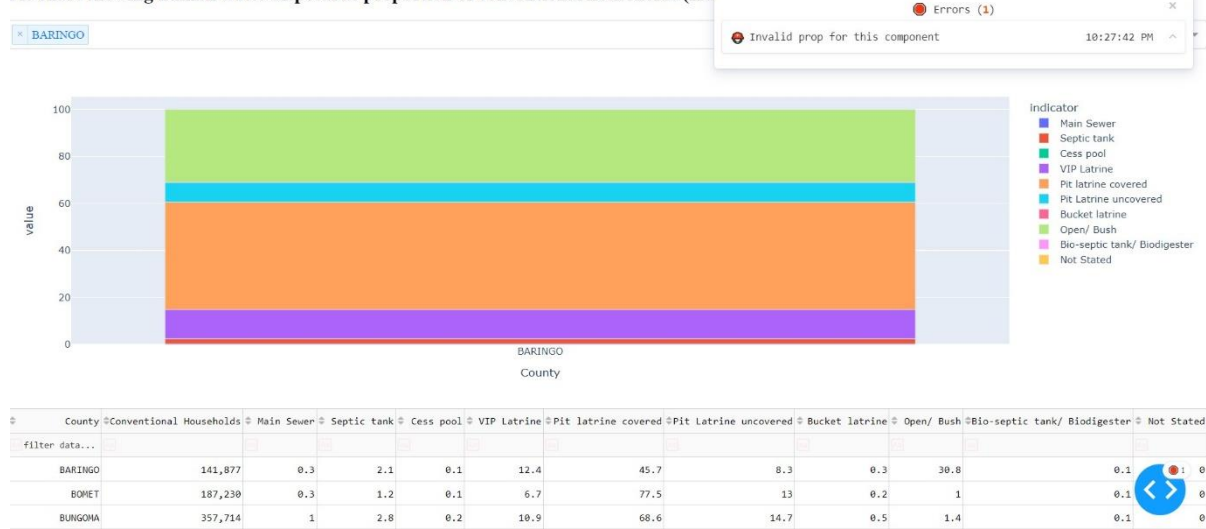
```
region_sales =
df.groupby(["Region", "Category"])["Sales"].sum().reset_index()

fig = px.bar(
    region_sales,
    x="Region", y="Sales",
    color="Category",
    title="Regional Sales by Category"
)
fig.show()
```

Expected Output:

- **Fig. 6.2:** Interactive Bar Chart – Regional Sales

Pie chart showing human waste disposal as proportion of conventional households (Ke...

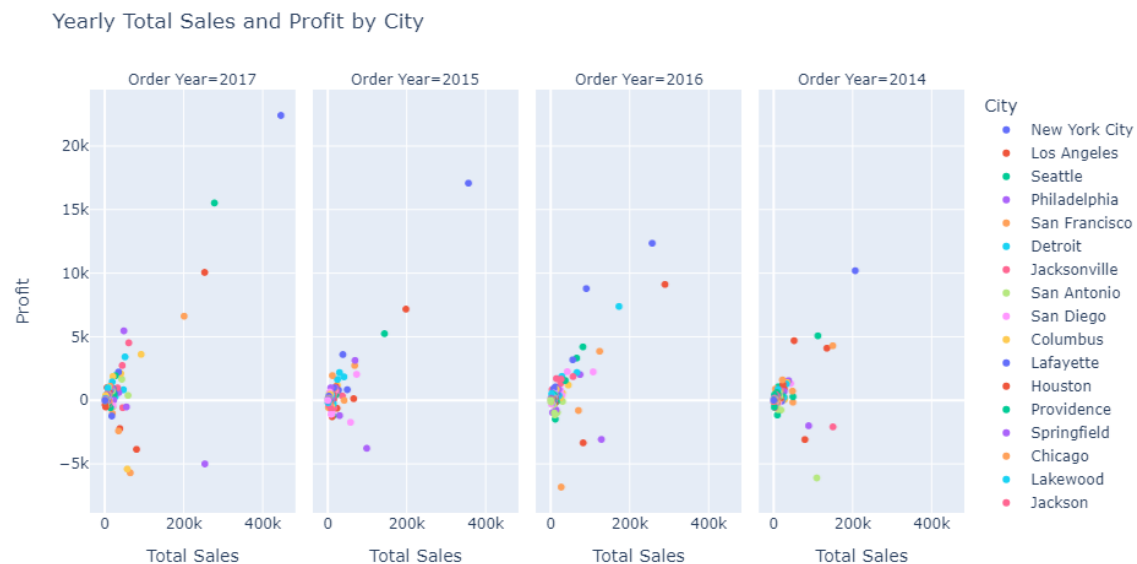


4. Interactive Scatter Plot

```
fig = px.scatter(  
    df, x="Sales", y="Profit",  
    color="Category",  
    size="Quantity",  
    title="Sales vs Profit"  
)  
fig.show()
```

Expected Output:

- **Fig. 6.3:** Interactive Scatter Plot – Sales vs Profit



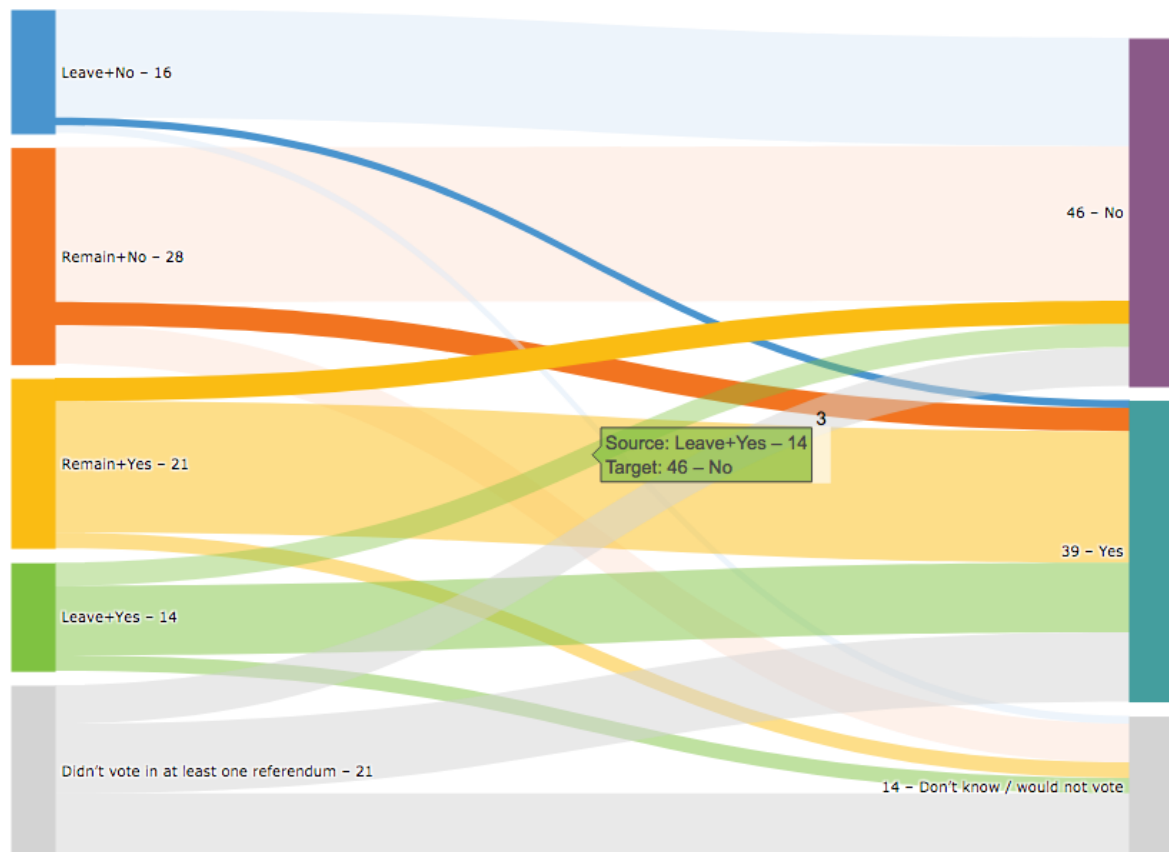
5. Sankey Diagram (Flow Visualization)

```
import plotly.graph_objects as go  
  
fig = go.Figure(go.Sankey(  
    node=dict(label=["West", "East", "Technology", "Furniture"]),  
    link=dict(  
        source=[0, 1, 0, 1],  
        target=[2, 2, 3, 3],  
        value=[50, 40, 30, 20]  
    )  
))  
fig.update_layout(title="Sales Flow: Region → Category")  
fig.show()
```

Expected Output:

- **Fig. 6.4:** Sankey Diagram – Region to Category Flow

Scottish Referendum Voters who now want Independence



RESULT & DISCUSSION

- Interactive plots allow deeper exploration using hover and zoom.
- Line charts reveal seasonal sales trends.
- Scatter plots highlight profit-loss anomalies.
- Sankey diagrams effectively show categorical flows.
- Interactive visuals enable users to explore data without static reports.

PROGRAM – 7

Hierarchical and Topographical Data Visualization in Tableau

AIM

To visualize hierarchical and geographical data in Tableau using treemaps, simulated sunburst charts, and maps.

THEORY

- **Hierarchical data** follows parent–child relationships (Category → Sub-Category → Product).
- **Treemaps** represent part-to-whole relationships using nested rectangles.
- **Sunburst charts** show hierarchy using concentric rings (simulated in Tableau).
- **Geospatial visualization** represents data on maps using geographic roles.

Map Types in Tableau:

- **Choropleth Map:** Color represents measure across regions.
- **Symbol Map:** Size and color represent measures at location level.

DATASET & TOOLS

- **Dataset:** *Sample – Superstore.xlsx*
- **Tool:** Tableau Desktop / Tableau Public

PROCEDURE (TABLEAU)

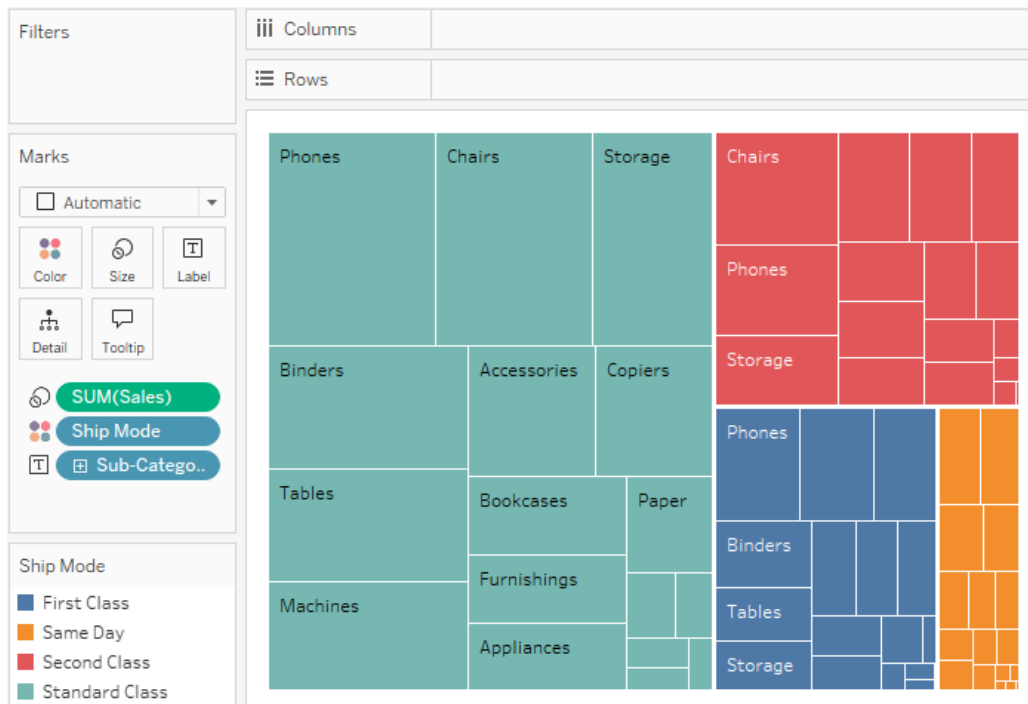
A. Hierarchical Visualization

1. Treemap (Sales by Category & Sub-Category)

1. Drag **Category** → **Rows**
2. Drag **Sub-Category** → **Color**
3. Drag **Sales** → **Size**
4. Click **Show Me** → **Treemap**

Expected Output:

- **Fig. 7.1:** Treemap – Sales by Category & Sub-Category

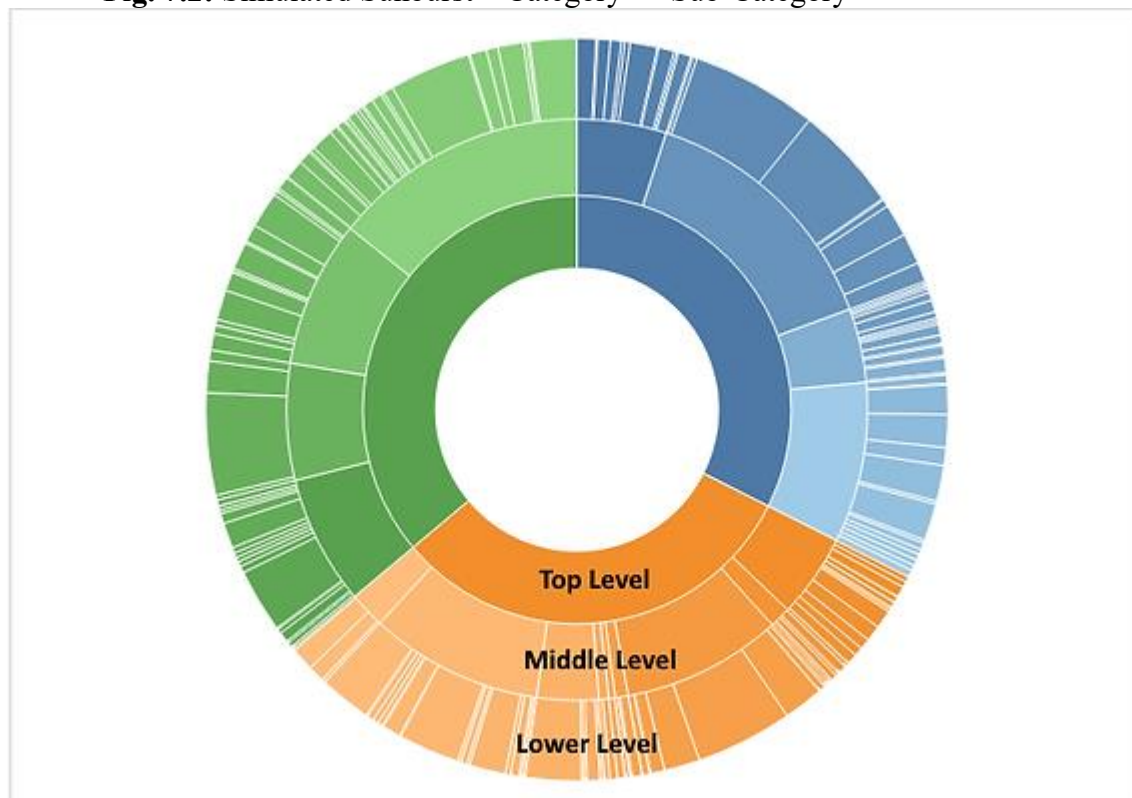


2. Simulated Sunburst Chart

- Create Pie chart:
 - Sales** → **Angle**
 - Category** → **Color**
- Duplicate chart for **Sub-Category**
- Combine both pie charts in a dashboard (inner = Category, outer = Sub-Category)

Expected Output:

- Fig. 7.2:** Simulated Sunburst – Category → Sub-Category



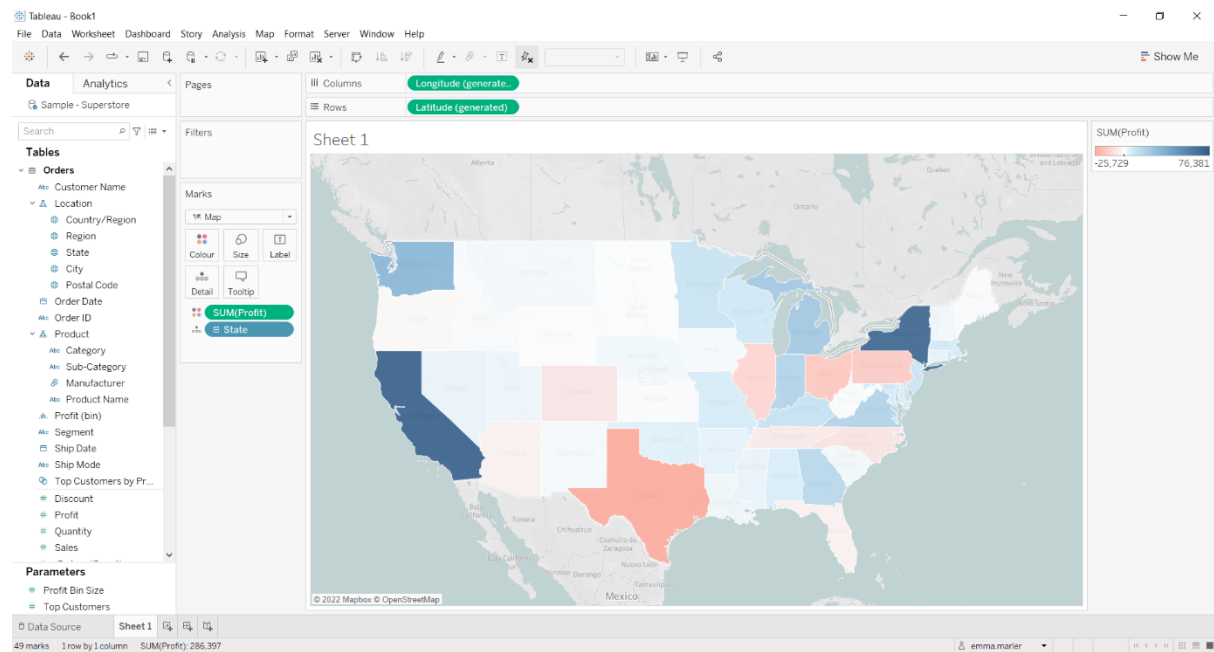
B. Topographical (Geospatial) Visualization

3. Choropleth Map (Profit by State)

1. Drag **State** → **View**
2. Change Marks → **Map**
3. Drag **Profit** → **Color**
4. Apply red–green color palette

Expected Output:

- **Fig. 7.3:** Choropleth Map – Profit by State

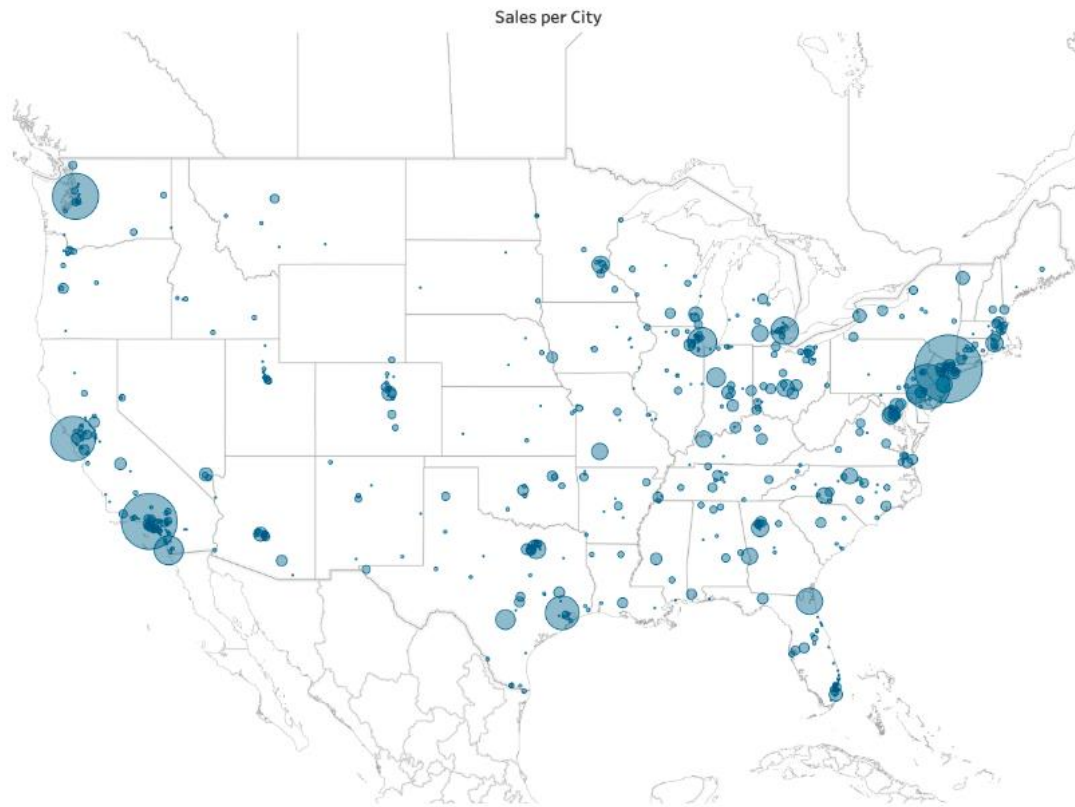


4. Symbol Map (Sales by City)

1. Drag **City** → **View**
2. Change Marks → **Circle**
3. Drag **Sales** → **Size**
4. Drag **Profit** → **Color**

Expected Output:

- **Fig. 7.4:** Symbol Map – Sales & Profit by City



RESULT & DISCUSSION

- Treemaps clearly show part-to-whole relationships.
- Simulated sunburst charts visualize hierarchical structure effectively.
- Choropleth maps highlight regional profit variations.
- Symbol maps reveal cities with high sales but low profit.
- Insights indicate margin issues in certain states and cities.

PROGRAM – 8

Calendar Heatmaps & Flow Data Visualization in Python

AIM

To visualize seasonality using calendar heatmaps and analyze categorical flows using Sankey diagrams in Python.

THEORY

- **Calendar Heatmap:** Displays daily values on a calendar layout, useful for identifying seasonality, weekday patterns, and anomalies.
- **Flow Visualization (Sankey):** Represents movement or transitions between categories, where link width indicates magnitude.
- Proper data preparation (date parsing, aggregation, node mapping) is essential for accurate visualization.

DATASET & TOOLS

- **Datasets:**
 - `daily_sales.csv` (date, value)
 - `taxi_sample.csv` (pickup_borough, dropoff_borough, payment_type, trips)
- **Tools:**
 - Python 3
 - Libraries: `pandas`, `matplotlib`, `plotly`, `calplot`

PROCEDURE (PYTHON)

A. Calendar Heatmap

1. Prepare Daily Data

```
import pandas as pd

df = pd.read_csv("daily_sales.csv", parse_dates=["date"])
df = df.set_index("date").asfreq("D").fillna(0)
```

2. Plot Calendar Heatmap

```
import calplot
import matplotlib.pyplot as plt

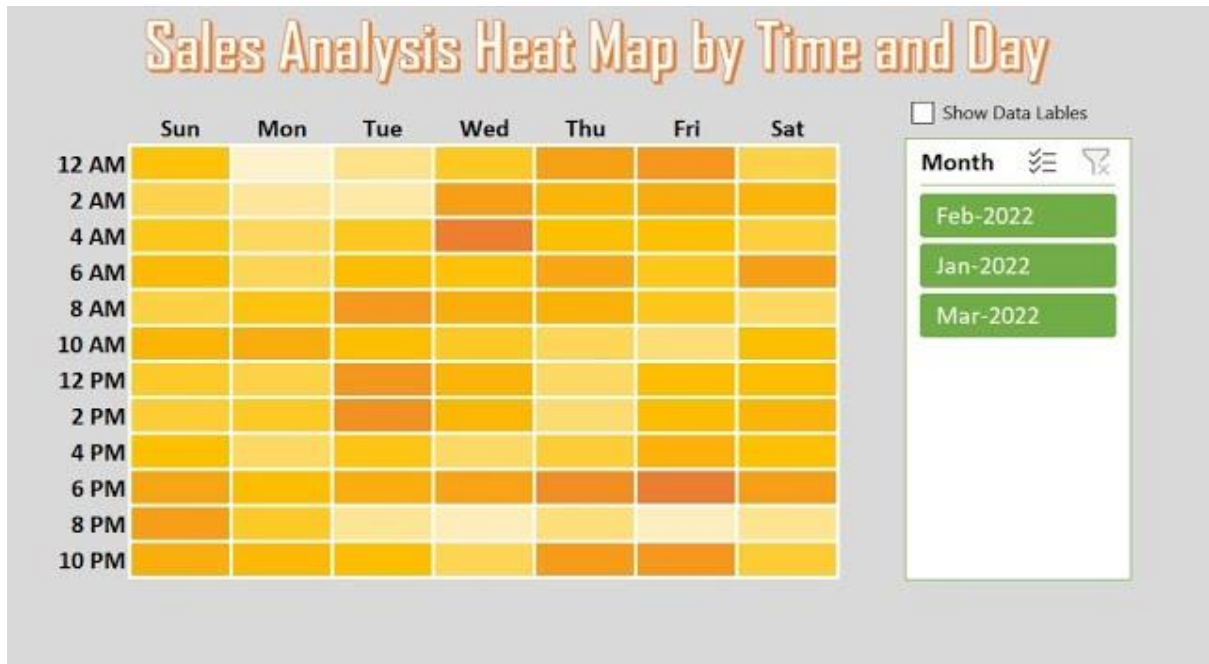
calplot.calplot(df['value'], cmap="YlOrRd")
plt.title("Daily Sales Calendar Heatmap")
plt.show()
```

Expected Output:

- **Fig. 8.1:** Calendar Heatmap – Daily Sales Intensity

Interpretation:

Darker days indicate higher sales; clusters reveal seasonal or weekend effects.



B. Flow Visualization (Sankey Diagram)

3. Prepare Flow Data

```
import plotly.graph_objects as go

labels = ["Manhattan", "Brooklyn", "Queens", "Card", "Cash"]
source = [0, 0, 1]
target = [1, 2, 3]
values = [1200, 800, 600]
```

4. Plot Sankey Diagram

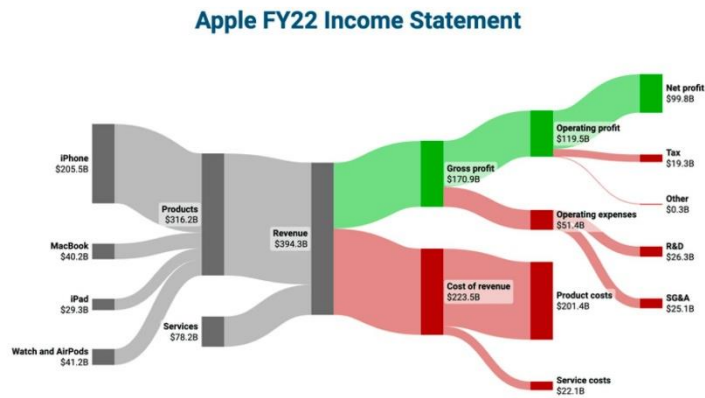
```
fig = go.Figure(go.Sankey(
    node=dict(label=labels),
    link=dict(source=source, target=target, value=values)
))
fig.update_layout(title="Flow: Pickup → Dropoff → Payment")
fig.show()
```

Expected Output:

- **Fig. 8.2:** Sankey Diagram – Flow Visualization

Interpretation:

Thicker links represent dominant flows, making multi-step relationships easy to identify.



RESULT & DISCUSSION

- Calendar heatmaps clearly highlight daily and seasonal patterns.
- Sankey diagrams effectively visualize multi-stage categorical flows.
- Interactive visuals allow quick identification of anomalies and dominant paths.
- Aggregation improves performance and clarity.

PROGRAM – 9

Time Series Data Visualization in Python

AIM

To visualize, analyze, and forecast time series data using Python by identifying trend, seasonality, and temporal dependence.

THEORY

- A time series consists of **trend**, **seasonality**, and **random noise**.
- **Stationarity** (constant mean and variance) is required for ARIMA modeling.
- **ACF** and **PACF** plots help identify AR and MA orders.
- **ARIMA/SARIMA** models are widely used for time series forecasting.
- Forecasts are accompanied by **confidence intervals** to express uncertainty.

DATASET & TOOLS

- **Dataset:** `air_passengers.csv` (Monthly passengers)
- **Tools:**
 - Python 3
 - **Libraries:** `pandas`, `matplotlib`, `statsmodels`, `plotly` (optional)

PROCEDURE (PYTHON)

1. Load and Visualize Time Series

```
import pandas as pd
import matplotlib.pyplot as plt

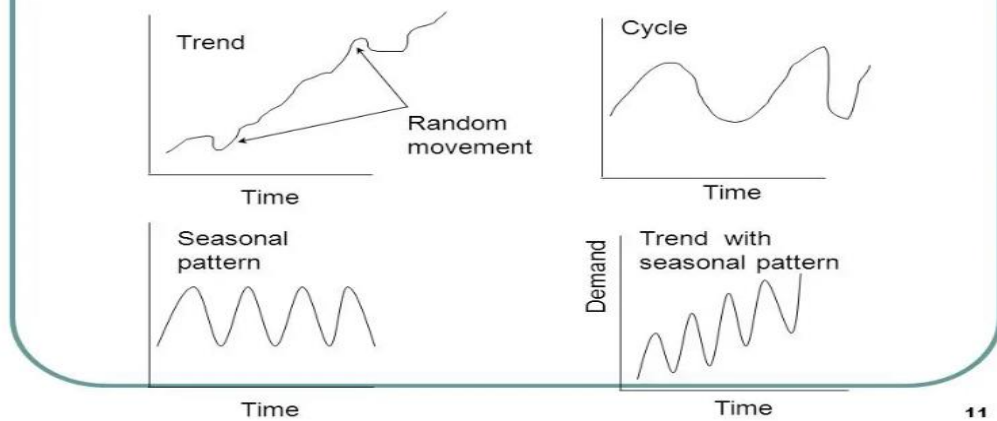
df = pd.read_csv("air_passengers.csv", parse_dates=["Month"],
index_col="Month")
ts = df['Passengers']

ts.plot(title="Monthly Airline Passengers")
plt.show()
```

Expected Output:

- **Fig. 9.1:** Time Series Line Plot
(Upward trend with yearly seasonality)

Time Series Components



11

2. Time Series Decomposition

```
from statsmodels.tsa.seasonal import seasonal_decompose
```

```
decomp = seasonal_decompose(ts, model="multiplicative", period=12)
decomp.plot()
plt.show()
```

Expected Output:

- **Fig. 9.2: Decomposition – Trend, Seasonal, Residual**



3. Stationarity Test (ADF)

```
from statsmodels.tsa.stattools import adfuller
```

```
adf = adfuller(ts.dropna())  
print("ADF p-value:", adf[1])
```

Interpretation:

p-value > 0.05 $\Rightarrow$ series is non-stationary.

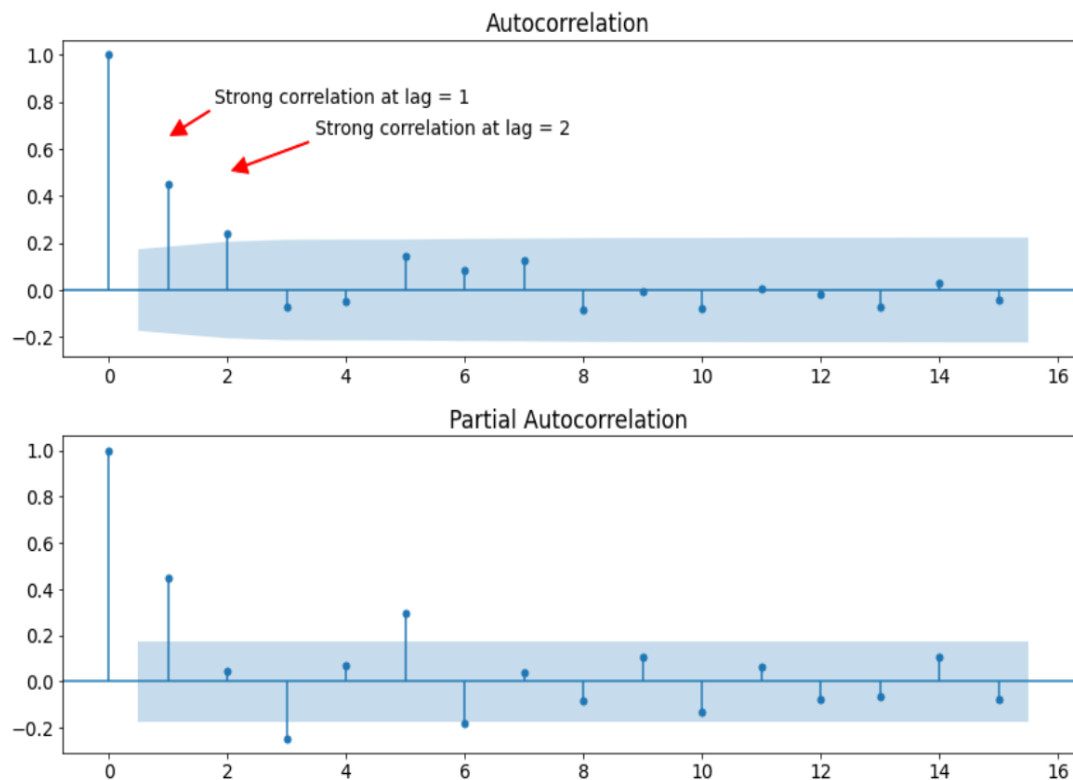
4. Differencing & ACF-PACF

```
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
```

```
ts_diff = ts.diff(12).diff().dropna()  
plot_acf(ts_diff); plot_pacf(ts_diff)  
plt.show()
```

Expected Output:

- **Fig. 9.3: ACF and PACF Plots**



5. SARIMA Model & Forecast

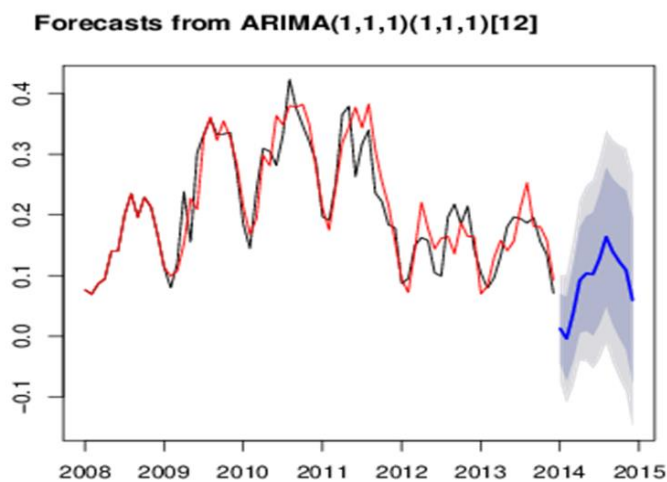
```
import statsmodels.api as sm  
  
model = sm.tsa.statespace.SARIMAX(  
    ts, order=(0,1,1), seasonal_order=(0,1,1,12)  
)  
results = model.fit()  
forecast = results.get_forecast(steps=12)  
forecast_ci = forecast.conf_int()
```


6. Forecast Visualization

```
ax = ts.plot(label="Observed")
forecast.predicted_mean.plot(ax=ax, label="Forecast")
ax.fill_between(
    forecast_ci.index,
    forecast_ci.iloc[:,0],
    forecast_ci.iloc[:,1],
    alpha=0.3
)
plt.legend()
plt.title("SARIMA Forecast")
plt.show()
```

Expected Output:

- **Fig. 9.4:** Forecast with Confidence Interval



RESULT & DISCUSSION

- Time series shows strong trend and seasonality.
- Differencing makes the series stationary.
- ACF/PACF guide ARIMA model selection.
- SARIMA provides reasonable forecasts with uncertainty bounds.
- Forecast confidence intervals widen with time horizon.

PROGRAM – 10

Dashboards, Actions, and Storytelling in Tableau

AIM

To build interactive dashboards in Tableau, apply actions, and present insights using story points for effective decision-making.

THEORY

- A **Dashboard** combines multiple visualizations into a single interactive view.
- **Actions** enable interaction across sheets:
 - **Filter Action:** filters other views based on selection
 - **Highlight Action:** emphasizes related data
 - **URL Action:** opens external links
 - **Parameter Action:** dynamically changes calculations
- **Storytelling** uses story points to present insights in a logical narrative.

Design Principles:

- Keep dashboards clean and uncluttered
- Use consistent colors and clear titles
- Limit to 3–5 key visuals
- Place KPIs at the top

DATASET & TOOLS

- **Dataset:** *Sample – Superstore.xlsx*
- **Tool:** Tableau Desktop / Tableau Public

PROCEDURE (TABLEAU)

1. Prepare Worksheets

Create the following worksheets:

- Bar Chart – Sales by Category
- Map – Profit by State
- Line Chart – Monthly Sales Trend
- KPI Tiles – Total Sales, Total Profit, Profit Ratio

2. Build Dashboard

1. Click **New Dashboard**.
2. Drag KPI tiles to the top.
3. Place Bar Chart and Map side by side.
4. Place Line Chart at the bottom.
5. Add filters (Category, Region).

Expected Output:

- **Fig. 10.1: Sales & Profit Dashboard**



3. Add Actions

Filter Action

- Clicking a category bar filters the map and line chart.

Highlight Action

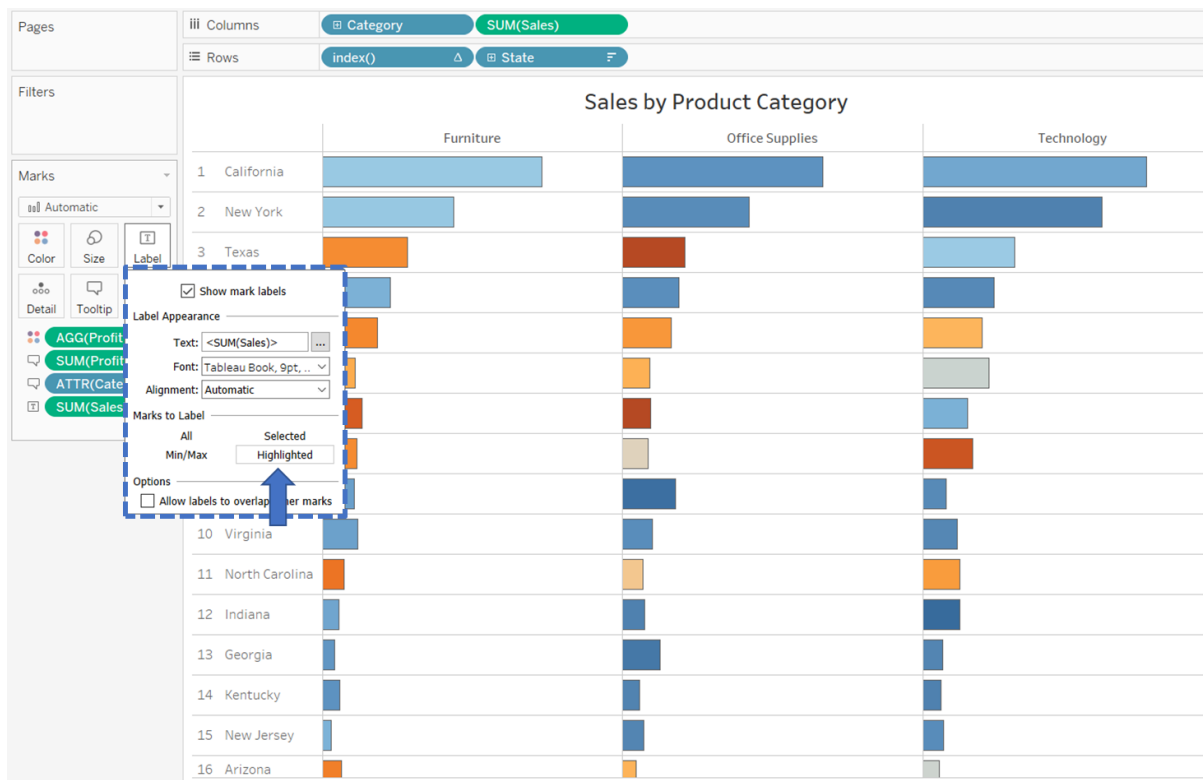
- Hovering over a category highlights related data.

URL Action (Optional)

- Clicking a state opens an external webpage.

Expected Output:

- Fig. 10.2:** Dashboard with Interactive Actions



4. Create a Story

- Click **New Story**.
- Add dashboards as story points.
- Add captions explaining insights.

Example story flow:

- Overall sales trend

2. Category-wise performance
3. Regional profit issues
4. Business recommendation

Expected Output:

- **Fig. 10.3:** Tableau Story with Story Points

AISD teacher turnover rates

Many Austin schools are still struggling to hold onto teachers, despite a multimillion-dollar effort, called Reach, to stem turnover in troubled schools since 2010. The graphics below show the turnover rate by year for each AISD school, along with how that rate compared with the district average for that year. Compiled from AISD statistics.

Austin American-Statesman

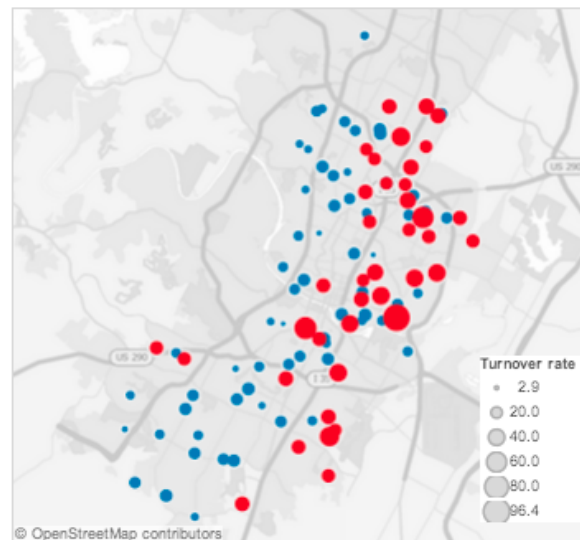
A data project produced by statesman.com
By Christian McDonald, with Beth Cortez-Neave

Use the slider to see rates for all campuses for each year

2013

Use the dropdown to see statistics for a single campus, by year

Mendez



Campus turnover rate compared to district rate Reach program schools

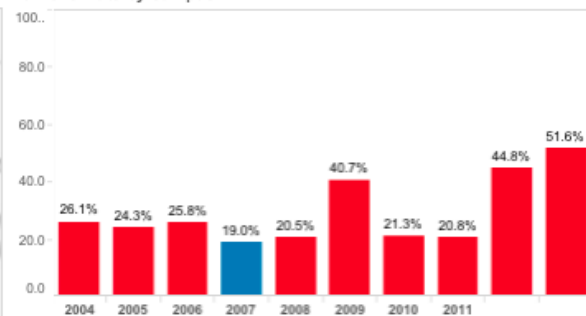
Yearly turnover rates are determined by the number of teachers that resign, retire, transfer to schools within the district or move to non-teaching jobs, divided by the total teachers per school.

Share

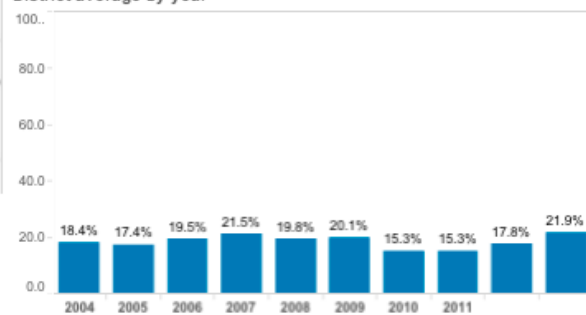
3,748 views

See more by this author

Turnover rate by campus



District average by year



Download

tableau

RESULT & DISCUSSION

- Dashboards provide a consolidated analytical view.
- Actions enable user-driven exploration.
- Story points convert analysis into clear narratives.
- In Superstore data, Technology leads sales, but some regions reduce profitability.